

DOCUMENTAÇÃO TÉCNICA

Manual do Signal Admin

Back-office de clientes e instalações Signal

Signal Admin 2026

Sumário

1. O que o Signal Admin faz	
2. Arquivos necessários para instalação	
3. Instalação por cópia do executável	
3.1. Escolha da pasta de instalação	
3.2. Build a partir do código-fonte	
4. Arquivo de configuração	
4.1. Modos de banco ('DB_MODE')	
4.2. Setup Turso do registry (produção)	
5. Variáveis de ambiente Turso (Platform API)	
6. Primeira inicialização	
7. Autenticação e navegação	
7.1. Login e logout	
7.2. Menu principal	
8. Usuários	
8.1. Criar usuário	
8.2. Editar usuário	
8.3. Excluir usuário	
9. Clientes	
9.1. Listagem e filtros	
9.2. Campos do formulário	
9.3. Criar e editar	
10. Provisionamento Turso	
10.1. Mensagens após a criação	
11. Deploy da Antena no Fly.io	
11.1. Pré-requisitos na máquina do operador	
11.2. O que o botão faz	
11.3. Limitações	
12. Exclusão de cliente	
13. Operação diária	
14. Solução de problemas	
14.1. Aplicativo não inicia / erro de banco	
14.2. Setup não aparece / login falha	
14.3. Cliente criado sem banco Turso	
14.4. Arquivos em `instals/` ausentes	

14.5. Falha ao instalar no Fly.io

14.6. CNPJ rejeitado

14.7. Não consigo excluir meu próprio usuário

15. Checklist de entrega

16. Apêndice: rotas HTTP

Este manual orienta a instalação, configuração e uso do Signal Admin, o back-office interno para gestão de clientes e provisionamento de instalações do ecossistema Signal. O público principal são operadores e responsáveis técnicos que cadastram clientes, geram bancos Turso e, opcionalmente, publicam a Antena no Fly.io.

1. O que o Signal Admin faz

O Signal Admin é um **aplicativo desktop** (webview + HTTP local) para administrar o registro de clientes Signal: cadastro, validade, status, provisionamento automático de banco Turso por cliente e deploy opcional da Antena no Fly.io.

Papéis no ecossistema:

```
Signal Admin → cria cliente + banco Turso + arquivos em instals/
               |
               v
Signal (site) → coleta eventos → sincroniza com o Turso do cliente
               |
               v
Antena        → visualiza eventos do Turso (local ou no Fly.io)
```

Aspecto	Signal	Antena	Signal Admin
Função	Coleta e grava eventos	Visualiza eventos	Cadastro e provisionamento
Público	Instalação no site	Operador / monitoramento	Back-office interno
Banco	Por instalação (Turso do cliente)	Lê o Turso do cliente	Registry próprio (<code>signal-admin</code>)
Autenticação	Registro de instalação	Login (setup)	Login (setup)

O Signal Admin **não coleta eventos** e **não substitui** o Signal nem a Antena. Ele prepara a infraestrutura e o cadastro comercial/técnico do cliente.

Ao iniciar, abre uma janela nativa com porta HTTP temporária em `127.0.0.1`. Não é necessário configurar porta nem abrir o navegador manualmente.

2. Arquivos necessários para instalação

Para instalar o Signal Admin em uma máquina de operação:

- o executável (`build/signal-admin` no Linux, `build/signal-admin.exe` no Windows);

- o arquivo de configuração `signal-admin.conf`.

Templates HTML, CSS e JavaScript ficam embutidos no executável. Não é necessário copiar a pasta `ui/` para a máquina final.

Use `sample.conf` na raiz do repositório como modelo para criar o `signal-admin.conf`.

Para provisionar bancos Turso e gerar configs de instalação, a máquina do operador também precisa das variáveis de ambiente `TURSO_ORG` e `TURSO_TOKEN` (ver seção 5). Para publicar a Antena no Fly.io, precisa do CLI `fly` / `flyctl` autenticado (ver seção 11).

3. Instalação por cópia do executável

3.1. Escolha da pasta de instalação

Crie uma pasta dedicada. Exemplos:

Linux:

```
/opt/signal-admin
```

Windows:

```
C:\SignalAdmin
```

Ambiente simples ou portátil (raiz do repositório clonado):

```
signal-admin/
```

Copie para essa pasta:

- `signal-admin` (Linux) ou `signal-admin.exe` (Windows);
- `signal-admin.conf`.

A pasta `installs/` será criada na raiz de trabalho do processo (em geral a pasta de onde o executável é iniciado, ou a raiz do projeto em desenvolvimento) quando um cliente for provisionado.

3.2. Build a partir do código-fonte

Requisitos: Go 1.25+ e CLI `tailwindcss`.

```
make build          # gera build/signal-admin
make tailwind-build # apenas CSS
go test ./...
```

Outros alvos:

```
make windows    # build/signal-admin.exe
make linux      # build/signal-admin_linux
make darwin     # build/signal-admin_mac
```

4. Arquivo de configuração

O aplicativo lê `signal-admin.conf` (Viper). Modelo em `sample.conf`:

```
DB_URL=libsql://signal-admin-registry-<org>.turso.io
DB_TOKEN=
DB_MODE=sync
DB_LOCAL_PATH=local.db
NOME_EMPRESA=Signal Admin
```

Chave	Função
<code>DB_URL</code>	URL do banco registry do Admin no Turso (libSQL)
<code>DB_TOKEN</code>	Token de acesso a esse banco
<code>DB_MODE</code>	<code>sync</code> , <code>remote</code> ou <code>local</code>
<code>DB_LOCAL_PATH</code>	Caminho do SQLite local (réplica ou banco puro)
<code>NOME_EMPRESA</code>	Nome exibido na interface

4.1. Modos de banco (`DB_MODE`)

Modo	Descrição
<code>sync</code>	Réplica local + Turso Cloud (padrão). Após cada escrita, Push + Pull.
<code>remote</code>	Conexão direta ao Turso Cloud.
<code>local</code>	SQLite local puro, sem rede. Ideal para desenvolvimento offline.

Em produção, use `DB_MODE=sync` com um banco Turso dedicado ao registry do Admin (ex.: `signal-admin-registry`). Esse banco é distinto dos bancos provisionados por cliente.

4.2. Setup Turso do registry (produção)

1. Criar banco Turso dedicado (ex.: `signal-admin-registry`).

2. Gerar token full-access.
3. Configurar `signal-admin.conf` com `DB_MODE=sync`.
4. Iniciar a aplicação — migrations locais e remotas + sync inicial.
5. Acessar o setup e criar o usuário administrador.

5. Variáveis de ambiente Turso (Platform API)

O provisionamento de **banco por cliente** usa a Platform API da organização Turso. Essas credenciais **não** vão no `signal-admin.conf`:

```
export TURSO_ORG=          # slug da organização Turso
export TURSO_TOKEN=        # token Platform API (distinto de DB_TOKEN)
```

Variável	Uso
<code>TURSO_ORG</code>	Organização onde os bancos dos clientes serão criados
<code>TURSO_TOKEN</code>	Token da Platform API da organização
<code>DB_TOKEN</code> (no conf)	Acesso ao registry do Admin — não substitui <code>TURSO_TOKEN</code>

Sem `TURSO_ORG` / `TURSO_TOKEN`, o cliente ainda pode ser cadastrado; o provisionamento Turso é ignorado e a interface exibe aviso.

6. Primeira inicialização

Execute:

```
./build/signal-admin
```

No primeiro acesso (sem usuários no banco), o sistema redireciona para `/setup`. Informe:

- nome;
- usuário (login);
- senha.

Após o setup, faça login. A sessão usa o mesmo padrão de autenticação do hs-financ (sessão, CSRF, bcrypt).

7. Autenticação e navegação

7.1. Login e logout

- **Login:** usuário e senha.
- **Logout:** ação disponível na interface (POST `/logout`).

7.2. Menu principal

Menu	Destino	Função
Cadastros → Clientes	<code>/config/clientes</code>	Listar, criar, editar e excluir clientes
Configuração → Usuários	<code>/config/usuarios</code>	Listar, criar, editar e excluir usuários admin

A home (`/`) funciona como hub com atalhos para Clientes e Usuários.

8. Usuários

Operadores do próprio Signal Admin (não confundir com usuários do Signal ou da Antena no site do cliente).

8.1. Criar usuário

Informe login, nome e senha. O login é definido na criação e **não pode ser alterado** depois.

8.2. Editar usuário

Na edição, apenas o **nome** pode ser alterado. Não há troca de senha pela tela de edição.

8.3. Excluir usuário

É possível excluir outros usuários. **Não é possível excluir o usuário da sessão atual** (mensagem de aviso).

9. Clientes

Cada cliente representa uma instalação Signal / um banco Turso associado.

9.1. Listagem e filtros

Em `/config/clientes` é possível filtrar por:

- código (`cliente_id`);

- nome;
- status.

A listagem é paginada. O badge **OnFly** indica se a Antena desse cliente já foi instalada no Fly.io.

9.2. Campos do formulário

Campo	Observação
Código (<code>cliente_id</code>)	Letras minúsculas e dígitos (<code>a-z0-9</code>). Imutável após a criação. Vira o nome do banco Turso.
Nome / Razão Social	Obrigatório
CNPJ	Validado (algoritmo de dígitos verificadores)
E-mail	Opcional
Telefone	Opcional
Validade (<code>valid_until</code>)	Data de validade da instalação/licença
Status	<code>active</code> , <code>suspended</code> ou <code>inactive</code>
Observação	Texto livre
OnFly	Somente leitura na UI; marcado após deploy bem-sucedido no Fly

9.3. Criar e editar

- **Novo:** `/config/clientes/new` — dispara o provisionamento Turso (seção 10).
- **Alterar:** `/config/clientes/{cliente_id}/edit` — atualiza dados; o código permanece fixo. Na mesma tela fica a seção Fly.io (seção 11).

10. Provisionamento Turso

Ao **criar** um cliente, se `TURSO_ORG` e `TURSO_TOKEN` estiverem definidos, o Admin:

1. Cria (ou detecta) o banco remoto com nome = `cliente_id` .
2. Aplica o schema Signal remoto.
3. Registra a licença.
4. Gera na pasta `instals/` :
 - `{cliente_id}-signal.conf`
 - `{cliente_id}-antena.conf`

Esses arquivos são usados na instalação do Signal no site e da Antena (local ou Fly).

10.1. Mensagens após a criação

Situação	Mensagem típica
Sucesso	Cliente criado e banco Turso provisionado.
Banco já existia	Cliente criado. Banco Turso já existia para este código.
Sem env Turso	Cliente criado (provisionamento Turso não configurado).
Falha parcial	Cliente criado, mas falha no provisionamento... (com detalhe)

O registro do cliente no Admin é gravado mesmo se o provisionamento falhar. Nesse caso, corrija a causa (credenciais, rede, permissões) e trate o banco/confs manualmente ou reexecute o fluxo operacional adequado.

11. Deploy da Antena no Fly.io

Na tela **Alterar Cliente**, a seção Fly.io exibe o botão **Instalar em fly.io** quando `onfly` é falso. Se já instalado, aparece apenas a confirmação "Já instalado no fly.io."

11.1. Pré-requisitos na máquina do operador

- CLI `fly` ou `flyctl` no `PATH`, autenticado (`fly auth login`);
- arquivo `instals/<cliente_id>-antena.conf` gerado pelo provisionamento Turso;
- rede para a API Fly e para o registry de imagens.

11.2. O que o botão faz

1. Cria o app `antena-<cliente_id>` na organização Fly `personal`.
2. Define secrets a partir de `instals/<cliente_id>-antena.conf`.
3. Faz deploy da imagem `ghcr.io/gapfranco/antena:latest` (modelo em `fly.toml.example`).
4. Marca `onfly = true` no cadastro do cliente.

A operação pode levar alguns minutos; a interface mostra indicador de progresso.

11.3. Limitações

- Não há botão de **destroy** / desinstalação do app Fly no Signal Admin.
- Se `onfly` já for verdadeiro, a instalação não é repetida ("Cliente já está instalado no fly.io").
- Falhas exibem flash com detalhe do erro.

12. Exclusão de cliente

Excluir um cliente remove **apenas o registro** no registry do Admin.

- O banco Turso do cliente na nuvem **permanece**.
- O app Fly `antena-<cliente_id>`, se existir, **não é destruído**.

Mensagens típicas:

- `Cliente excluído.`
- `Cliente excluído. O banco Turso "<codigo>" continua existente na nuvem.`

13. Operação diária

Checklist sugerido para o operador:

1. Confirmar que `signal-admin.conf` e (se for provisionar) `TURSO_ORG` / `TURSO_TOKEN` estão corretos.
2. Abrir o Admin e autenticar.
3. Cadastrar ou atualizar clientes (código, validade, status).
4. Verificar a mensagem de provisionamento e a pasta `instals/`.
5. Entregar `{cliente_id}-signal.conf` à equipe que instala o Signal no site.
6. Se a Antena for publicada na nuvem: autenticar no `fly`, abrir o cliente e usar **Instalar em fly.io**.
7. Conferir o badge OnFly e o status do app no painel Fly.

14. Solução de problemas

14.1. Aplicativo não inicia / erro de banco

- Verifique `DB_URL`, `DB_TOKEN` e `DB_MODE` no `signal-admin.conf`.
- Em desenvolvimento sem rede, use `DB_MODE=local`.
- Confirme permissão de escrita em `DB_LOCAL_PATH`.

14.2. Setup não aparece / login falha

- Setup só ocorre quando não há usuários. Se o banco já tem admin, use login.
- Confirme que está usando o mesmo arquivo de banco/conf da instalação.

14.3. Cliente criado sem banco Turso

- Exporte `TURSO_ORG` e `TURSO_TOKEN` **antes** de iniciar o processo.
- Lembre: `DB_TOKEN` do conf não substitui `TURSO_TOKEN`.

- Verifique permissões do token Platform API na organização.

14.4. Arquivos em `instals/` ausentes

- Provisionamento precisa ter concluído com sucesso.
- Confirme o diretório de trabalho ao iniciar o executável (a pasta `instals/` fica relativa a ele).

14.5. Falha ao instalar no Fly.io

- `fly auth whoami` — sessão válida?
- `fly / flyctl` no `PATH`?
- Existe `instals/<cliente_id>-antena.conf`?
- Nome do app `antena-<cliente_id>` já em uso por outra conta/org?

14.6. CNPJ rejeitado

- O formulário valida dígitos verificadores. Confira o número informado.

14.7. Não consigo excluir meu próprio usuário

- Comportamento esperado. Peça a outro admin ou use outra sessão.

15. Checklist de entrega

Antes de considerar uma instalação de cliente concluída no Admin:

- ☐ Cliente cadastrado com código estável (`a-z0-9`)
- ☐ Status e validade corretos
- ☐ Banco Turso provisionado (ou decisão consciente de pular)
- ☐ Arquivos em `instals/{codigo}-signal.conf` e `instals/{codigo}-antena.conf`
- ☐ Config Signal entregue à equipe de campo
- ☐ Se aplicável: Antena no Fly.io e `onfly` marcado
- ☐ Operador ciente de que exclusão no Admin **não** apaga Turso nem Fly

16. Apêndice: rotas HTTP

Rotas relevantes da interface local (protegidas após autenticação, exceto setup/login):

Método	Rota	Função
GET/POST	/setup	Primeiro administrador
GET/POST	/login	Autenticação
POST	/logout	Encerrar sessão
GET	/	Home
GET	/config/clientes	Lista de clientes
GET/POST	/config/clientes/new	Novo cliente (+ provisionamento)
GET/POST	/config/clientes/{cliente_id}/edit	Alterar cliente
POST	/config/clientes/{cliente_id}/fly	Instalar Antena no Fly.io
POST	/config/clientes/{cliente_id}/delete	Excluir registro do cliente
GET	/config/usuarios	Lista de usuários
GET/POST	/config/usuarios/new	Novo usuário
GET/POST	/config/usuarios/{id}/edit	Alterar usuário
POST	/config/usuarios/{id}/delete	Excluir usuário

Estáticos da UI: GET /static/... (embutidos no binário).